

TEAM 2C1

Database Design Document

MoMo SMS Data Processing System

Team Member	Role
Therese Mary Nshimyumurwa	Team Lead
Aimable Bancunguye	Backend and Database
Eloi Mizero	ETL and Testing
Davy Dushimiyimana	Frontend and Integration
Clive Tanaka Mushipe	Documentation and QA

Week 2 - Database Design and Implementation

BSE Program | Cohort 1

1. Entity Relationship Diagram (ERD)

1.1 ERD Diagram

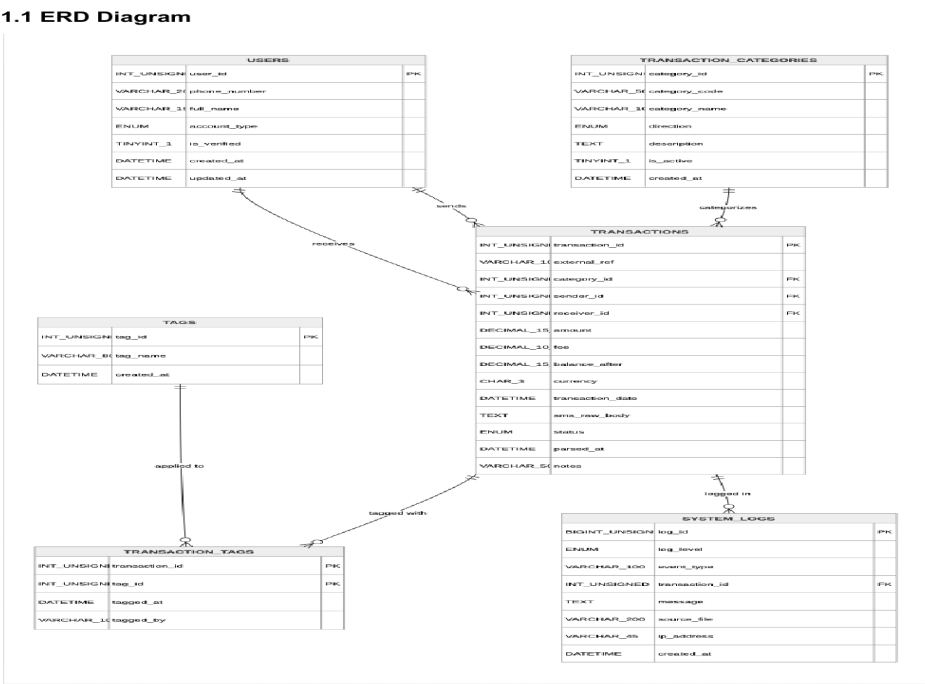


Figure 1: Entity Relationship Diagram for the MoMo SMS Data Processing System

1.2 Design Rationale

The database schema was designed around the structure of MoMo SMS transaction data and the need to store, query, and analyse that data reliably over time. Six tables were identified from the business requirements and the XML data format. TRANSACTION_CATEGORIES separates transaction types from the main transaction records. This avoids repeating category names in every row and makes it easy to add or rename categories without touching transaction data. One category can apply to many transactions, giving a one-to-many relationship with TRANSACTIONS. USERS stores every person or entity involved in a transaction, whether they are the sender or the receiver. Rather than creating two separate tables for senders and receivers, we used a single USERS table and linked it to TRANSACTIONS via two foreign keys: sender_id and receiver_id. This keeps the schema clean and avoids data duplication. TRANSACTIONS is the central table. It holds all parsed SMS data including amount, fee, balance after the transaction, currency, transaction date, the original SMS body, and processing status. Storing the raw SMS body supports auditing and reprocessing if the parser has errors later. TAGS and TRANSACTION_TAGS were added to support flexible labelling of transactions. Because one transaction can carry multiple tags and one tag can apply to many transactions, a direct link between the two tables would not work. TRANSACTION_TAGS is a junction table that resolves this many-to-many relationship into two one-to-many relationships. This is the most important structural decision in the schema. SYSTEM_LOGS records every event produced by the ETL pipeline. Logs can optionally reference a specific transaction, allowing developers to trace errors back to the exact record that caused them. One

transaction can produce many log entries, another one-to-many relationship. DECIMAL(15,2) was chosen for all monetary columns instead of FLOAT because floating-point arithmetic introduces rounding errors that are unacceptable in financial data. The utf8mb4 character set was used throughout to correctly store SMS content that may include special characters or non-ASCII text.

2. Data Dictionary

All tables, columns, data types, constraints, and descriptions for the MoMo SMS database.

2.1 Transactions Table

[illegible]

2.2 Users Table

Column Name	Data Type	Constraint	Description
user_id	INT UNSIGNED	PK, NOT NULL, AUTO_INCREMENT	Auto-incremented unique identifier for each user record
phone_number	VARCHAR(20)	UNIQUE, NOT NULL	MoMo-registered phone number used as the primary account identifier
full_name	VARCHAR(150)	NOT NULL	Full name of the MoMo account holder as extracted from SMS records
account_type	ENUM	DEFAULT 'individual'	Classification of the account: individual, merchant, or agent
is_verified	TINYINT(1)	DEFAULT 0	Boolean flag: 1 means the user has been verified, 0 means not yet
created_at	DATETIME	DEFAULT NOW()	Timestamp of when the user record was first created in the database
updated_at	DATETIME	DEFAULT NOW()	Timestamp of the most recent update made to the user record

2.3 Transaction_Categories Table

Column Name	Data Type	Constraint	Description
category_id	INT UNSIGNED	PK, NOT NULL, AUTO_INCREMENT	Auto-incremented unique identifier for each transaction category
category_code	VARCHAR(50)	UNIQUE, NOT NULL	Short alphanumeric code used in processing logic (e.g. TRF, PAY, ...)

category_name	VARCHAR(100)	UNIQUE, NOT NULL	Human-readable name of the category (e.g. Transfer, Payment, Debit)
direction	ENUM	NOT NULL	Indicates money flow relative to the sender: INBOUND or OUTBOUND
description	TEXT	NULL	Detailed explanation of what transaction types fall under this category
is_active	TINYINT(1)	DEFAULT 1	Flag to soft-delete categories without removing historical transactions
created_at	DATETIME	DEFAULT NOW()	Timestamp of when the category was first added to the database

2.4 Tags Table

Column Name	Data Type	Constraint	Description
tag_id	INT UNSIGNED	PK, NOT NULL, AUTO_INCREMENT	Auto-incremented unique identifier for each tag
tag_name	VARCHAR(80)	UNIQUE, NOT NULL	The label text for the tag (e.g. flagged, high-value, recurring)
created_at	DATETIME	DEFAULT NOW()	Timestamp of when the tag was created in the system

2.5 Transaction_Tags Table (Junction Table)

Column Name	Data Type	Constraint	Description
transaction_id	INT UNSIGNED	PK, FK, NOT NULL	Foreign key referencing TRANSACTIONS; part of the composite primary key
tag_id	INT UNSIGNED	PK, FK, NOT NULL	Foreign key referencing TAGS; part of the composite primary key
tagged_at	DATETIME	DEFAULT NOW()	Timestamp of when the tag was applied to the transaction
tagged_by	VARCHAR(100)	NULL	Name or identifier of the person or process that applied the tag

This table resolves the many-to-many relationship between TRANSACTIONS and TAGS. The composite primary key (transaction_id + tag_id) ensures the same tag cannot be applied to the same transaction more than once.

2.6 System_Logs Table

Column Name	Data Type	Constraint	Description
log_id	BIGINT UNSIGNED	PK, NOT NULL, AUTO_INCREMENT	Auto-incremented unique identifier for each log entry
log_level	ENUM	NOT NULL	Severity classification: INFO, WARNING, ERROR, or CRITICAL
event_type	VARCHAR(100)	NOT NULL	Short label for the type of event (e.g. SMS_PARSED, DB_INSERT)
transaction_id	INT UNSIGNED	FK, NULL	Optional foreign key linking the log entry to a specific transaction
message	TEXT	NOT NULL	Detailed description of the event, error, or action recorded in this log
source_file	VARCHAR(200)	NULL	Name of the script or module that generated the log entry (e.g. parser.py)
ip_address	VARCHAR(45)	NULL	IP address of the machine or service that triggered the logged event
created_at	DATETIME	DEFAULT NOW()	Timestamp of when the log entry was automatically generated by the system

3. Sample Queries and Results

The SQL script was run in MySQL Workbench against the momo_sms_db database. The following queries demonstrate the database working correctly with real data.

Query 1: View all transactions

```
SELECT * FROM transactions LIMIT 0, 1000;
```

This retrieves all transaction records. The result shows 8 rows with all columns including the external reference, amount, fee, balance, SMS body, and status.

transaction_id	external_ref	category_id	sender_id	receiver_id	amount	fee	balance_after	currency	transaction_date	sms_raw_body	status
1	TXND0250501001	1	2	1	30000.00	0.00	150000.00	RWF	2025-05-01 08:15:00	You have received 30,000 RWF from Amable B...	SUCCESS
2	TXND0250502001	3	1	6	20000.00	100.00	129900.00	RWF	2025-05-02 10:30:00	Payment of 20,000 RWF to Kigali Mart Ltd was...	SUCCESS
3	TXND0250503001	4	1	NULL	50000.00	0.00	124900.00	RWF	2025-05-03 12:00:00	Airtime purchase of 5,000 RWF was successful...	SUCCESS
4	TXND0250504001	5	NULL	3	100000.00	0.00	20000.00	RWF	2025-05-04 09:30:00	Cash in: 100,000 RWF deposited by agent. Bal...	SUCCESS
5	TXND0250505001	7	2	NULL	30000.00	0.00	169800.00	RWF	2025-05-05 14:20:00	Cash Out: 30,000 RWF withdrawn. Fee: 200 R...	SUCCESS
6	TXND0250506001	8	4	NULL	15000.00	0.00	109800.00	RWF	2025-05-06 07:45:00	Utility payment of 15,000 RWF was processed...	SUCCESS
7	TXND0250507001	9	NULL	5	250000.00	0.00	45000.00	RWF	2025-05-07 11:00:00	International remittance of 250,000 RWF receiv...	SUCCESS
8	TXND0250508001	2	3	4	10000.00	100.00	189800.00	RWF	2025-05-08 16:10:00	Transfer of 10,000 RWF to Isidore Duchesneym...	FAILED

Figure 2: Result of SELECT query on the transactions table showing 8 rows returned in 0.0047 seconds

Query 2: View all users

```
SELECT * FROM users;
```

This retrieves all user records stored in the database, confirming that sender and receiver accounts are correctly stored with their phone numbers and account types.

[Screenshot: Result of SELECT * FROM users]

Query 3: View transactions with category names (JOIN)

```
SELECT t.transaction_id, t.external_ref, c.category_name, t.amount, t.status FROM  
transactions t JOIN transaction_categories c ON t.category_id = c.category_id;
```

This JOIN query links each transaction to its category name, demonstrating referential integrity between the transactions and transaction_categories tables.

[Screenshot: Result of JOIN query]

Query 4: Count transactions by status

```
SELECT status, COUNT(*) AS total FROM transactions GROUP BY status;
```

This aggregation query groups transactions by their processing status, giving a summary of how many records are marked SUCCESS, FAILED, or PENDING.

4. Security and Unique Rules

Several constraints and design decisions were added specifically to protect data integrity and prevent incorrect or duplicate records from entering the database.

4.1 UNIQUE Constraints

Rule	Table	Column(s)	Purpose
Phone number must be unique	USERS	phone_number	Prevents two user accounts from sharing the same Mobile Number
Category code must be unique	TRANSACTION_CATEGORIES	category_code	Ensures processing logic always maps to exactly one category
Category name must be unique	TRANSACTION_CATEGORIES	category_name	Prevents duplicate category labels that would confuse merchants
Tag name must be unique	TAGS	tag_name	Prevents identical tags being created under different IDs
Tag per transaction is unique	TRANSACTION_TAGS	transaction_id + tag_id	Composite PK prevents the same tag being applied twice to a transaction

4.2 CHECK Constraints

Rule	Table	Column	Condition
Amount cannot be negative	TRANSACTIONS	amount	amount >= 0
Fee cannot be negative	TRANSACTIONS	fee	fee >= 0
Balance cannot be negative	TRANSACTIONS	balance_after	balance_after >= 0

4.3 Foreign Key Constraints with Cascade Rules

Relationship	ON DELETE Rule	Reason
TRANSACTIONS.category_id -> TRANSACTION_CATEGORIES	RESTRICT	Prevent deleting a category that has transactions linked to it
TRANSACTIONS.sender_id -> USERS	RESTRICT	Prevent deleting a user who has transaction history
TRANSACTIONS.receiver_id -> USERS	SET NULL	Receiver can be deleted if they are no longer in the system
TRANSACTION_TAGS.transaction_id -> TRANSACTIONS	CASCADE	When a transaction is deleted, remove all its tag associations
TRANSACTION_TAGS.tag_id -> TAGS	CASCADE	When a tag is deleted, remove all its associations with transactions
SYSTEM_LOGS.transaction_id -> TRANSACTIONS	SET NULL	Logs are kept even if the linked transaction is removed

4.4 Additional Security Measures

The database uses the utf8mb4 character set and utf8mb4_unicode_ci collation throughout. This ensures that SMS content containing special characters, currency symbols, or non-ASCII text is stored and compared correctly without data loss or encoding errors. A verified merchants view (v_verified_merchants) was created to expose only confirmed merchant accounts to application queries, limiting the surface area of data visible to front-end services. A daily summary view (v_daily_summary) aggregates transaction totals by date, meaning the raw transaction table never needs to be exposed directly for reporting purposes. The sms_raw_body column stores the original unmodified SMS text alongside parsed data. This allows any parsing errors to be caught and corrected by reprocessing the original input, providing an audit trail for every record in the database. Strategic indexes were placed on phone_number (USERS), transaction_date (TRANSACTIONS), and category_id (TRANSACTIONS) to ensure queries on the most commonly filtered columns run efficiently as the dataset grows.

MySQL Workbench interface showing the **momo_sms_db** schema with all 6 tables and 2 views created successfully.

The **SCHEMAS** panel on the left lists the following objects:

- md_water_services
- momo_sms_db
 - Tables
 - system_logs
 - tags
 - transaction_catego...
 - transactions
 - transaction_tags
 - users
 - Views
 - v_daily_summary
 - v_verified_mercha...
 - Stored Procedures
 - Functions
 - sys

The **Query 1** panel shows the following SQL query:

```
SELECT * FROM transactions LIMIT 0, 1000
```

The **Result Grid** displays the following data:

transaction...	external_ref	category_id	tender_id	receiver_id	amount	fee	balance_after	currency	transaction_date	sms_raw_body	status
1	TXN20250501001	1	2	1	50000.00	0.00	150000.00	RWF	2025-05-01 08:15:00	You have received 50,000 RWF from Amabile B.	SUCCESS
2	TXN20250502001	3	1	6	20000.00	1.00	129900.00	RWF	2025-05-02 10:30:00	Payment of 20,000 RWF to Ngizi Mart Ltd was...	SUCCESS
3	TXN20250503001	4	1	3	50000.00	0.00	124900.00	RWF	2025-05-03 12:00:00	Airtime purchase of 5,000 RWF was successful.	SUCCESS
4	TXN20250504001	8	3	3	100000.00	0.00	200000.00	RWF	2025-05-04 09:00:00	Cash in: 100,000 RWF deposited by agent. Bal...	SUCCESS
5	TXN20250505001	7	2	3	30000.00	0.00	169800.00	RWF	2025-05-05 14:20:00	Cash Out: 30,000 RWF withdrawn. Fee: 200 R...	SUCCESS
6	TXN20250506001	8	4	3	150000.00	0.00	109800.00	RWF	2025-05-06 07:45:00	Utility payment of 15,000 RWF was processed...	SUCCESS
7	TXN20250507001	9	3	5	250000.00	0.00	450000.00	RWF	2025-05-07 11:00:00	International remittance of 250,000 RWF receiv...	SUCCESS
8	TXN20250508001	12	3	4	10000.00	150.00	189900.00	RWF	2025-05-08 18:15:00	Transfer of 10,000 RWF to Davy Duchinyemen...	FAILED

The **Action Output** panel shows the following information:

- Time: 20:22:02
- Action: SELECT * FROM transactions LIMIT 0, 1000
- Response: 8 row(s) returned
- Duration / Fetch Time: 0.0047 sec / 0.00002...

Figure 3: MySQL Workbench showing the momo_sms_db schema with all 6 tables and 2 views created successfully

5. AI Usage Statement

In completing this assignment, the team used AI assistance for grammar and syntax checking in written documentation, and to verify SQL syntax in the database setup script. AI was not used to generate the ERD design, the database relationships, or the technical explanations. All design decisions, table structures, and justifications reflect the team's own analysis of the MoMo SMS data requirements.

SQL-to-JSON Mapping Summary

The JSON schemas in `examples/json_schemas.json` represent the same entities defined in the SQL schema. Each SQL table maps directly to a JSON object or array. Relational links that use foreign keys in SQL are represented as nested objects in JSON. For example, a transaction record in SQL stores only a `category_id` integer, while the equivalent JSON object nests the full category object inside the transaction, which is the format an API response would return to a front-end client.